

insurance_model.py

October 6, 2026

```
[1]: from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator
import pandas as pd

# =====
# 1. CREATE SPARK SESSION
# =====

spark = SparkSession.builder \
    .appName("Insurance Claim Prediction") \
    .getOrCreate()

print("Spark Version:", spark.version)
```

Spark Version: 3.5.3

```
[2]: # =====
# 2. LOAD DATASET FROM HDFS
# =====

df = spark.read.option("header", "true") \
    .option("inferSchema", "true") \
    .csv("hdfs://namenode:9000/insurance/data/Insurance.csv")

print("Dataset loaded successfully")
```

Dataset loaded successfully

```
[3]: # =====
# 3. DISPLAY DATA
# =====

df.show(10)
```

```
+---+---+-----+-----+-----+-----+-----+
|age|sex|  bmi|children|smoker|region|  charges|claim|
+---+---+-----+-----+-----+-----+-----+-----+
```

19	0	27.9	0	1	3	16884.924	1
18	1	33.77	1	0	2	1725.5523	1
28	1	33.0	3	0	2	4449.462	0
33	1	22.705	0	0	1	21984.47061	0
32	1	28.88	0	0	1	3866.8552	1
31	0	25.74	0	0	2	3756.6216	0
46	0	33.44	1	0	2	8240.5896	1
37	0	27.74	3	0	1	7281.5056	0
37	1	29.83	2	0	0	6406.4107	0
60	0	25.84	0	0	1	28923.13692	0

only showing top 10 rows

```
[4]: # =====
# 4. CHECK SCHEMA
# =====

df.printSchema()
```

```
root
|-- age: integer (nullable = true)
|-- sex: integer (nullable = true)
|-- bmi: double (nullable = true)
|-- children: integer (nullable = true)
|-- smoker: integer (nullable = true)
|-- region: integer (nullable = true)
|-- charges: double (nullable = true)
|-- claim: integer (nullable = true)
```

```
[5]: # =====
# 5. CHECK NUMBER OF ROWS AND COLUMNS
# =====

print("Number of rows:", df.count())
print("Number of columns:", len(df.columns))

print("Columns:")
print(df.columns)
```

Number of rows: 1338

Number of columns: 8

Columns:

['age', 'sex', 'bmi', 'children', 'smoker', 'region', 'charges', 'claim']

```
[6]: # =====
# 6. CHECK MISSING VALUES
```

```
# =====

from pyspark.sql.functions import col, sum

print("Missing values:")

df.select([
    sum(col(c).isNull().cast("int")).alias(c)
    for c in df.columns
]).show()
```

Missing values:

```
+---+---+---+---+---+---+---+---+
|age|sex|bmi|children|smoker|region|charges|claim|
+---+---+---+---+---+---+---+---+
| 0| 0| 0|      0|      0|      0|      0|  0|
+---+---+---+---+---+---+---+---+
```

```
[7]: # =====
# 7. SUMMARY STATISTICS
# =====

df.describe().show()
```

```
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+
|summary|          age|          sex|          bmi|
children|        smoker|        region|        charges|
claim|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+
| count|          1338|          1338|          1338|
1338|          1338|          1338|          1338|
1338|
|  mean| 39.20702541106129|0.5052316890881914|30.663396860986538|
1.0949177877429|0.20478325859491778|
1.515695067264574|13270.422265141257|0.5852017937219731|
| stddev|14.049960379216147|0.5001595692843767| 6.098186911679012|1.205492739781
914|0.40369403754561717|1.1048849185826901|12110.011236693992|0.4928713935274979
|
|   min|          18|          0|          15.96|
0|          0|          0|          1121.8739|          0|
|   max|          64|          1|          53.13|
5|          1|          3|          63770.42801|          1|
+-----+-----+-----+-----+-----+
```

-----+-----+-----+-----+
+

```
[8]: # =====  
# 8. FEATURE ASSEMBLY  
# =====  
  
feature_cols = [  
    'age',  
    'sex',  
    'bmi',  
    'children',  
    'smoker',  
    'region',  
    'charges'  
]  
  
assembler = VectorAssembler(  
    inputCols=feature_cols,  
    outputCol="features"  
)  
  
feature_df = assembler.transform(df)
```

```
[9]: # =====  
# 9. SELECT FEATURES AND TARGET  
# =====  
  
final_df = feature_df.select(  
    "features",  
    "claim"  
)  
  
final_df.show(10)
```

```
+-----+-----+  
|          features|claim|  
+-----+-----+  
|[19.0,0.0,27.9,0...|    1|  
|[18.0,1.0,33.77,1...|    1|  
|[28.0,1.0,33.0,3...|    0|  
|[33.0,1.0,22.705,...|    0|  
|[32.0,1.0,28.88,0...|    1|  
|[31.0,0.0,25.74,0...|    0|  
|[46.0,0.0,33.44,1...|    1|  
|[37.0,0.0,27.74,3...|    0|  
|[37.0,1.0,29.83,2...|    0|
```

```
|[60.0,0.0,25.84,0...|    0|
+-----+-----+
only showing top 10 rows
```

```
[10]: # =====
# 10. TRAIN-TEST SPLIT
# =====

train_data, test_data = final_df.randomSplit(
    [0.8, 0.2],
    seed=42
)

print("Training records:", train_data.count())
print("Testing records:", test_data.count())
```

Training records: 1108
Testing records: 230

```
[11]: # =====
# 11. LOGISTIC REGRESSION MODEL
# =====

lr = LogisticRegression(
    labelCol="claim",
    featuresCol="features",
    maxIter=10
)

model = lr.fit(train_data)

print("Logistic Regression model trained successfully")
```

Logistic Regression model trained successfully

```
[12]: # =====
# 12. MAKE PREDICTIONS
# =====

predictions = model.transform(test_data)

predictions.select(
    "claim",
    "probability",
    "prediction"
).show(20)
```

```
+-----+-----+-----+-----+
```

claim	probability	prediction
1 [0.39108508000139...		1.0
1 [0.27099841801650...		1.0
1 [0.14992209144910...		1.0
1 [0.24662582038716...		1.0
0 [0.54240060312067...		0.0
1 [0.01684559889045...		1.0
1 [0.17793813946757...		1.0
1 [0.09173861146248...		1.0
0 [0.49463464764458...		1.0
1 [0.11472266980487...		1.0
1 [0.06401255089662...		1.0
1 [0.13697756795595...		1.0
1 [0.17596099498626...		1.0
1 [0.25416639216053...		1.0
0 [0.52128804762427...		0.0
0 [0.48376911277876...		1.0
0 [0.20968207602073...		1.0
1 [0.05536533455266...		1.0
1 [0.43177402757451...		1.0
1 [0.06994274569300...		1.0

only showing top 20 rows

```
[13]: # =====
# 13. MODEL EVALUATION - AUC
# =====

evaluator = BinaryClassificationEvaluator(
    labelCol="claim",
    metricName="areaUnderROC"
)

auc = evaluator.evaluate(predictions)

print(f"Test Area Under ROC (AUC): {auc:.4f}")
```

Test Area Under ROC (AUC): 0.9270

```
[14]: # =====
# 14. FEATURE COEFFICIENTS
# =====

coefficients = model.coefficients.toArray()

coef_df = pd.DataFrame({
```

```

        "Feature": feature_cols,
        "Coefficient": coefficients
    })

    print("Feature Coefficients:")

    print(
        coef_df.sort_values(
            by="Coefficient",
            ascending=False
        )
    )

```

Feature Coefficients:

	Feature	Coefficient
4	smoker	4.131033e+00
2	bmi	2.616304e-01
0	age	2.619465e-02
6	charges	6.752251e-07
5	region	-4.890564e-02
1	sex	-1.309535e-01
3	children	-1.392408e+00

```

[15]: predictions.select(
        "claim",
        "probability",
        "prediction"
    ).write.mode("overwrite").parquet(
        "hdfs://namenode:9000/user/student/insurance/results/predictions.parquet"
    )

    print("Predictions saved successfully to HDFS")

```

Predictions saved successfully to HDFS

```

[16]: saved_predictions = spark.read.parquet(
        "hdfs://namenode:9000/user/student/insurance/results/predictions.parquet"
    )

    saved_predictions.show(20, truncate=False)

    saved_predictions.select(
        "claim",
        "prediction"
    ).show(20)

```

```

+-----+-----+-----+-----+
|claim|probability|prediction|
+-----+-----+-----+-----+

```

1	[0.3910850800013902,0.6089149199986098]	1.0	
1	[0.2709984180165057,0.7290015819834943]	1.0	
1	[0.14992209144910612,0.8500779085508938]	1.0	
1	[0.2466258203871636,0.7533741796128364]	1.0	
0	[0.542400603120672,0.45759939687932805]	0.0	
1	[0.016845598890453103,0.9831544011095469]	1.0	
1	[0.17793813946757822,0.8220618605324218]	1.0	
1	[0.09173861146248513,0.9082613885375148]	1.0	
0	[0.49463464764458254,0.5053653523554175]	1.0	
1	[0.11472266980487064,0.8852773301951293]	1.0	
1	[0.06401255089662214,0.9359874491033778]	1.0	
1	[0.13697756795595084,0.8630224320440492]	1.0	
1	[0.17596099498626425,0.8240390050137357]	1.0	
1	[0.2541663921605309,0.7458336078394692]	1.0	
0	[0.5212880476242759,0.47871195237572406]	0.0	
0	[0.48376911277876955,0.5162308872212305]	1.0	
0	[0.20968207602073954,0.7903179239792605]	1.0	
1	[0.055365334552660954,0.9446346654473391]	1.0	
1	[0.4317740275745182,0.5682259724254818]	1.0	
1	[0.06994274569300817,0.9300572543069918]	1.0	

+-----+-----+-----+-----+

only showing top 20 rows

+-----+-----+-----+-----+
claim prediction
+-----+-----+-----+-----+
1 1.0
1 1.0
1 1.0
1 1.0
0 0.0
1 1.0
1 1.0
1 1.0
0 1.0
1 1.0
1 1.0
1 1.0
1 1.0
1 1.0
1 1.0
0 0.0
0 1.0
0 1.0
1 1.0
1 1.0
1 1.0

+-----+-----+-----+-----+

only showing top 20 rows

```
[17]: # =====  
# 16. STOP SPARK  
# =====  
  
spark.stop()
```